

AWS EKS Secrets Architecture — IRSA + Secrets Store CSI Driver + Secrets Manager + SSM Parameter Store

1. Goal

We wanted to build a production-grade Kubernetes secret management architecture on AWS using:

- Amazon EKS
- IRSA (IAM Roles for Service Accounts)
- AWS Secrets Manager
- AWS Systems Manager Parameter Store
- Secrets Store CSI Driver
- AWS Provider for CSI
- RDS PostgreSQL

The final goal was:

- No hardcoded secrets
- No AWS access keys inside pods
- No Kubernetes Secret objects for sensitive values
- Runtime-only secret injection
- Secure IAM-based access control
- Enterprise-grade architecture

2. Final Architecture

```
Application Pod
  ↓
Kubernetes ServiceAccount
  ↓
IRSA (OIDC federation)
  ↓
STS AssumeRoleWithWebIdentity
  ↓
Temporary AWS credentials
  ↓
Secrets Store CSI Driver
  ↓
AWS Provider
```

↓
Secrets Manager + Parameter Store
↓
Mounted files inside pod

3. AWS Services Used

Service	Purpose
EKS	Kubernetes cluster
IAM	Authorization
IRSA	Pod-level AWS authentication
STS	Temporary credentials
Secrets Manager	Sensitive secrets
SSM Parameter Store	Configuration values
KMS	Encryption
CSI Driver	Mount secrets into pod filesystem
RDS PostgreSQL	Database

4. Secrets Manager vs Parameter Store

This is a VERY important distinction.

Secrets Manager

Used for:

- Passwords
- Tokens
- API keys
- DB credentials
- Rotated secrets

Supports:

- automatic rotation
- versioning

- high security integrations

Example:

```
{  
  "username": "postgres",  
  "password": "mypassword",  
  "dbname": "appdb"  
}
```

Our secret:

```
prod/app/postgres
```

Parameter Store

Used for:

- app environment
- feature flags
- hostnames
- log levels
- non-sensitive configs

Examples:

```
/prod/app/APP_ENV  
/prod/app/DB_HOST  
/prod/app/LOG_LEVEL
```

5. Why Use Full Paths Like `/prod/app/LOG_LEVEL`

This is hierarchical naming.

Example:

```
/prod/app/LOG_LEVEL
```

Breakdown:

Part	Meaning
/prod	environment
/app	application
LOG_LEVEL	actual parameter

Benefits:

- clean organization
- easy filtering
- environment isolation
- IAM scoping
- easier automation

Large companies may use:

```
/prod/payments/api/DB_HOST
/prod/payments/api/LOG_LEVEL
/dev/orders/backend/DB_HOST
```

This becomes critical at scale.

6. Why We Did NOT Put Everything in Secrets Manager

Not everything is a secret.

Example:

```
DB_HOST
APP_ENV
LOG_LEVEL
```

These are configuration values.

Best practice:

Data Type	Store
sensitive	Secrets Manager

Data Type	Store
non-sensitive	Parameter Store

This separation:

- reduces cost
- improves clarity
- simplifies rotation
- follows security principles

7. What Is IRSA

IRSA = IAM Roles for Service Accounts

It allows Kubernetes pods to securely access AWS services WITHOUT static AWS credentials.

8. How IRSA Works

Traditional Bad Approach

```
AWS_ACCESS_KEY_ID
AWS_SECRET_ACCESS_KEY
```

inside pods.

Problems:

- static credentials
- rotation pain
- insecure
- credential leakage risk

IRSA Approach

```
ServiceAccount
  ↓
OIDC Token
  ↓
```

```
AWS STS
  ↓
temporary credentials
```

No static credentials exist.

9. OIDC Provider

IRSA requires an OIDC provider.

We discovered:

```
https://oidc.eks.ap-southeast-1.amazonaws.com/id/
C02A48890839C1A0B4BB8D0084FB54C4
```

AWS IAM trusts this OIDC issuer.

This creates trust between:

- Kubernetes
- AWS IAM

10. IRSA Trust Policy

Critical policy:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",

      "Principal": {
        "Federated": "arn:aws:iam::<ACCOUNT_ID>:oidc-provider/oidc.eks.ap-
southeast-1.amazonaws.com/id/C02A48890839C1A0B4BB8D0084FB54C4"
      },

      "Action": "sts:AssumeRoleWithWebIdentity",

      "Condition": {
```

```
    "StringEquals": {
      "oidc.eks.ap-southeast-1.amazonaws.com/id/
C02A48890839C1A0B4BB8D0084FB54C4:sub":
        "system:serviceaccount:default:app-sa"
    }
  }
}
```

Important:

```
system:serviceaccount:default:app-sa
```

means ONLY:

- namespace: default
- service account: app-sa

can assume the role.

This is extremely secure.

11. Service Account Annotation

IRSA requires annotation.

```
annotations:
  eks.amazonaws.com/role-arn: arn:aws:iam::<ACCOUNT_ID>:role/app-irsa-role
```

This annotation is what activates IRSA.

Without this: IRSA does not work.

12. CSI Driver

CSI = Container Storage Interface

Secrets Store CSI Driver allows external secret systems to mount secrets as files inside pods.

This means:

```
Secrets Manager  
→ files inside container
```

without application code talking to AWS APIs.

13. Why CSI Is Powerful

Applications only read files:

```
with open("/etc/secrets/DB_PASSWORD") as f:  
    password = f.read()
```

No:

- AWS SDK
- secret API calls
- IAM code

This decouples applications from cloud providers.

14. SecretProviderClass (SPC)

This is the most important Kubernetes object in this setup.

It tells CSI:

- which secrets to fetch
 - from where
 - how to transform them
-

15. Final Working SecretProviderClass

```
apiVersion: secrets-store.csi.x-k8s.io/v1  
kind: SecretProviderClass  
  
metadata:
```



```
name: aws-test-secret
namespace: default

spec:
  provider: aws

parameters:
  objects: |
    - objectName: "prod/app/postgres"
      objectType: "secretsmanager"
      jmesPath:
        - path: "username"
          objectAlias: "DB_USERNAME"

        - path: "password"
          objectAlias: "DB_PASSWORD"

        - path: "dbname"
          objectAlias: "DB_NAME"

    - objectName: "/prod/app/DB_HOST"
      objectType: "ssmparameter"
      objectAlias: "DB_HOST"

    - objectName: "/prod/app/APP_ENV"
      objectType: "ssmparameter"
      objectAlias: "APP_ENV"

    - objectName: "/prod/app/LOG_LEVEL"
      objectType: "ssmparameter"
      objectAlias: "LOG_LEVEL"
```

16. What Is jmesPath

This is VERY important.

Secrets Manager stores JSON blobs:

```
{
  "username": "postgres",
  "password": "mypassword",
```

```
"dbname": "appdb"
}
```

jmesPath extracts individual keys.

Example:

```
- path: "username"
  objectAlias: "DB_USERNAME"
```

This creates file:

```
/etc/secrets/DB_USERNAME
```

without exposing entire JSON blob.

17. Why jmesPath Matters

Without jmesPath:

```
/etc/secrets/prod_app_postgres
```

contains full JSON.

With jmesPath:

```
/etc/secrets/DB_USERNAME
/etc/secrets/DB_PASSWORD
```

Cleaner and safer.

18. Why We Saw prod_app_postgres

AWS provider automatically mounts:

- original full secret object AND

- extracted jmesPath files

So both existed.

19. Major Problems We Faced

Problem 1 — Pod Identity Confusion

Initially we used:

- EKS Pod Identity

instead of:

- IRSA

Problem:

- CSI ecosystem is still more mature with IRSA
- logs became confusing
- auth partially worked
- mount failures persisted

Important lesson:

supported != operationally mature

Problem 2 — Wrong Secret ARN

IAM policy allowed:

prod/app/postgres

But SPC referenced:

rds!db-xxxx

Result:

- permission mismatch

- confusing errors
-

Problem 3 — Wrong jmesPath Keys

We initially used:

```
db_host  
db_name
```

But actual secret contained:

```
host  
dbname
```

jmesPath is exact-match only.

Problem 4 — Wrong SecretProviderClass Name

Deployment used:

```
secretProviderClass: "aws-app-secrets-test"
```

But actual SPC name was:

```
aws-test-secret
```

Tiny naming mismatch caused major confusion.

Very common Kubernetes issue.

Problem 5 — YAML Formatting

SPC embeds YAML inside YAML:

```
objects: |
```

Indentation matters heavily.

Blank lines and formatting mistakes can silently fail.

20. Why IRSA Was Better Here

IRSA is:

- older
- battle-tested
- heavily used by CSI ecosystem
- deeply integrated into AWS examples

Pod Identity is newer and still evolving.

21. Final Working Deployment

Pod used:

```
serviceAccountName: app-sa
```

with:

```
volumeMounts:  
- mountPath: /etc/secrets
```

and CSI volume.

Secrets appeared as local files.

22. Final Mounted Files

Inside pod:

```
/etc/secrets
```

contained:

```
APP_ENV
DB_HOST
DB_NAME
DB_PASSWORD
DB_USERNAME
LOG_LEVEL
```

This proved:

- IRSA working
- CSI working
- Secrets Manager working
- Parameter Store working
- jmesPath working

23. ESO vs CSI Driver

This is VERY important.

ESO = External Secrets Operator

ESO syncs external secrets INTO Kubernetes Secrets.

Flow:

```
Secrets Manager
→ ESO
→ Kubernetes Secret
→ Pod
```

CSI Driver Flow

```
Secrets Manager
→ CSI
→ mounted runtime files
→ Pod
```

24. Key Difference — Security

ESO

Secrets become Kubernetes Secret objects.

Meaning:

- stored in etcd
- base64 encoded
- visible through Kubernetes API

Even if encrypted: still broader exposure.

CSI Driver

Secrets are:

- runtime mounted only
- NOT stored in etcd
- NOT Kubernetes Secret objects

Much stronger security posture.

25. When To Use ESO

ESO is easier when:

- apps expect env vars
 - legacy apps
 - simpler operations
 - GitOps-heavy environments
-

26. When To Use CSI

CSI is better when:

- high security
- avoid etcd secret storage
- runtime-only secrets

- cloud-native workloads
 - strict compliance
-

27. Which Approach Is Best

There is NO universal best.

Best Security

IRSA + CSI

Best Simplicity

IRSA + ESO

28. Recommended Enterprise Approach

For production:

Use:

- IRSA
 - Secrets Store CSI Driver
 - Secrets Manager
 - Parameter Store
 - KMS CMKs
 - VPC endpoints
-

29. Future Improvements

Add:

- secret rotation
- automatic pod reload
- VPC endpoints
- namespace isolation
- least-privilege IAM

- KMS key policies
 - multi-env architecture
-

30. Final Important Lessons

Kubernetes problems are often:

- naming mismatches
 - YAML formatting
 - IAM scoping
 - identity federation
 - tiny configuration mistakes
-

AWS IAM debugging is usually:

NOT:

missing permission

BUT:

wrong resource ARN

Distributed systems debugging principle

Reduce complexity until:

- smallest working system exists

Then gradually reintroduce complexity.

This is exactly how senior platform engineers debug real production systems.